

Web Application with Hibernate (using Annotation)

Example 1:

In this article, we are building a simple **User Management** web application using JSP, Servlet, and Hibernate. This web application manages a collection of **users** with the basic feature: list, insert, update, delete (or CRUD operations - Create, Update, Read and Delete).

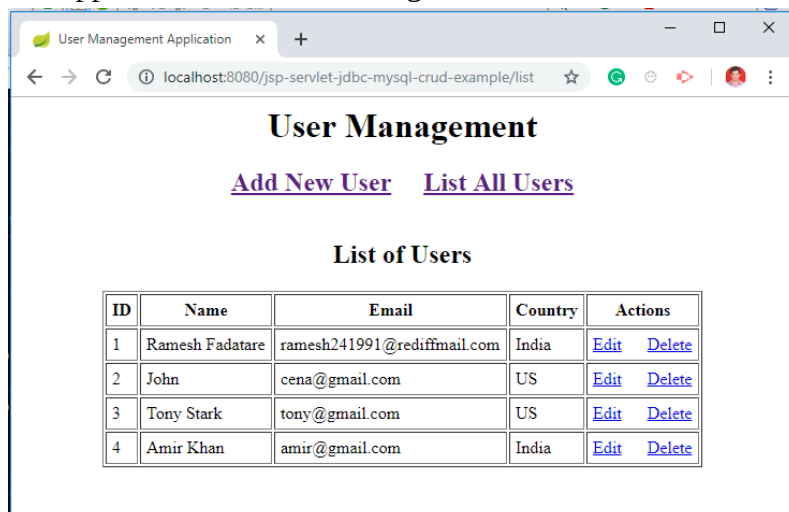
In this article, we will use [Hibernate Java-based configuration](#) without using *hibernate.cfg.xml* to connect **MySQL** database.

JSP Servlet Hibernate Web Application

We will develop below simple basic features in our **User Management** web application:

1. Create a User
2. Update a User
3. Delete a User
4. Retrieve a User
5. List of all Users

The application looks something like this:



Development Steps

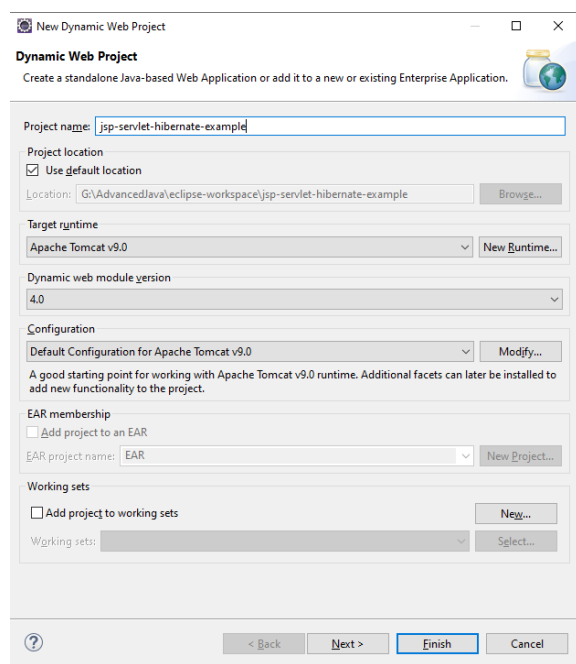
1. Create an Eclipse Dynamic Web Project
2. Add Dependencies
3. Project Structure
4. ORACLE Database Setup
5. Create a JPA entity - User.java
6. Hibernate Java-based configuration

7. Create a UserDAO.java
8. Create a UserServicelet.java
9. Creating User Listing JSP Page - user-list.jsp
10. Create a User Form JSP Page - user-form.jsp
11. Creating Error JSP page
12. Deploying and Testing the Application Demo

Let's begin the Journey:

To create a new dynamic Web project in Eclipse:

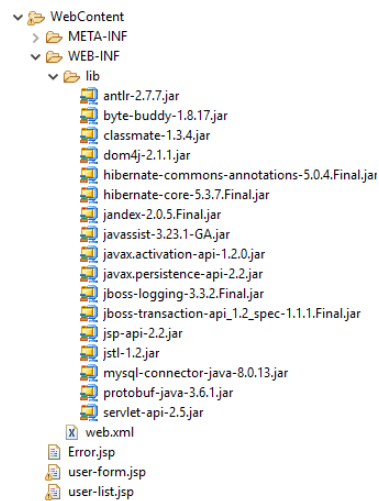
1. On the main menu select **File > New > Dynamic Web Project**.



2. Enter project name as "jsp-servlet-hibernate-example";
3. Make sure that the target runtime is set to Apache Tomcat with the currently supported version.
4. Click **Next** and **Finish**.

2. Add Dependencies

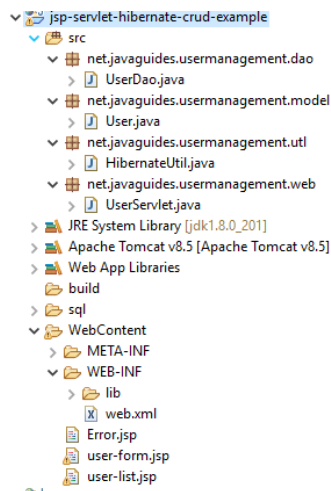
Add the latest release of below jar files to the lib folder:



You can download all these JAR files from my GitHub repository - [jsp-servlet-hibernate-mysql-tutorial](#)

3. Project Structure

Standard project structure for your reference -



4. Oracle Database Setup

Now, create a **users** table using below DDL script:

```
CREATE TABLE users (
    id NUMBER(3) PRIMARY KEY,
    name VARCHAR2(120) NOT NULL,
```

```
email VARCHAR2(220) NOT NULL,  
country VARCHAR2(120)  
);
```

You can use either SQL Plus Command Line Client or _____ to create the table. The above a **users** table looks like this:

```
SQL> desc users;  
Name                               Null?    Type  
-----  
ID                                 NOT NULL NUMBER(3)  
NAME                             NOT NULL VARCHAR2(120)  
EMAIL                             NOT NULL VARCHAR2(220)  
COUNTRY                           VARCHAR2(120)  
  
SQL>
```

5. Create a JPA Entity - User.java

Let's create a *User* persistent class that is mapped to a database table - users.

A simple Persistent class should follow some rules:

- **A no-arg constructor:** It is recommended that you have a default constructor with at least package visibility so that hibernate can create the instance of the Persistent class by *newInstance()* method.
- **Provide an identifier property:** It is better to assign an attribute as an id. This attribute behaves as a primary key in a database.
- **Declare getter and setter methods:** The Hibernate recognizes the method by getter and setter method names by default.
- **Prefer non-final class:** Hibernate uses the concept of proxies, which depends on the persistent class. The application programmer will not be able to use proxies for lazy association fetching.

```
package com.company.usermanagement.model;
```

```
import javax.persistence.Column;  
import javax.persistence.Entity;  
import javax.persistence.GeneratedValue;  
import javax.persistence.GenerationType;  
import javax.persistence.Id;  
import javax.persistence.Table;
```

```
@Entity
```

```

@Table(name="instructor")
public class User {
    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    @Column(name="id")
    protected int id;
    @Column(name="name")
    protected String name;
    @Column(name="email")
    protected String email;
    @Column(name="country")
    protected String country;
    public User() {
    }
    public User(String name, String email, String country) {
        super();
        this.name = name;
        this.email = email;
        this.country = country;
    }
    public User(int id, String name, String email, String country) {
        super();
        this.id = id;
        this.name = name;
        this.email = email;
        this.country = country;
    }
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }
    public String getCountry() {

```

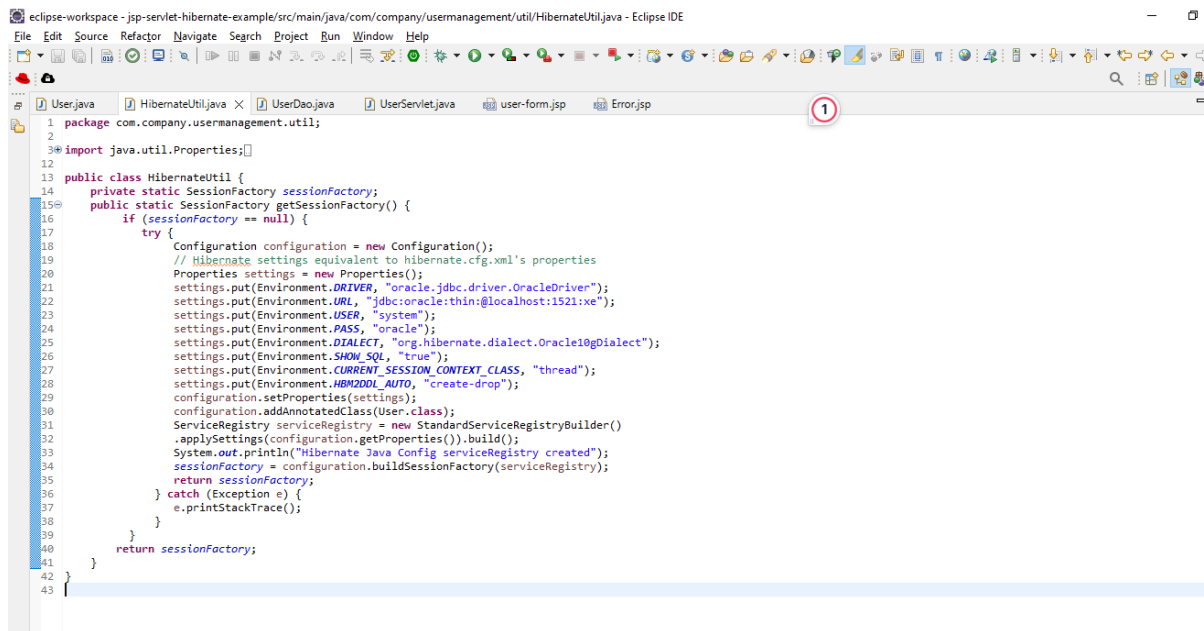
```

        return country;
    }
    public void setCountry(String country) {
        this.country = country;
    }
}

```

6. Hibernate Java-based configuration

Below diagram shows Java code for Hibernate settings equivalent to hibernate.cfg.xml's properties:



- ServiceRegistry holds the services that Hibernate will need during bootstrapping and at runtime.
- StandardServiceRegistryBuilder - Builder for standard ServiceRegistry instances.
- HibernateUtil - This is a helper class to bootstrap hibernate SessionFactory. In most Hibernate applications, the SessionFactory should be instantiated once during application initialization. The single instance should then be used by all code in a particular process, and any Session should be created using this single SessionFactory. The SessionFactory is thread-safe and can be shared; a Session is a single-threaded object.

Here is the complete code:

```
package com.company.usermanagement.util;
```

```

import java.util.Properties;

import com.company.usermanagement.model.User;

import org.hibernate.SessionFactory;
import org.hibernate.boot.registry.StandardServiceRegistryBuilder;
import org.hibernate.cfg.Configuration;
import org.hibernate.cfg.Environment;
import org.hibernate.service.ServiceRegistry;

public class HibernateUtil {
    private static SessionFactory sessionFactory;
    public static SessionFactory getSessionFactory() {
        if (sessionFactory == null) {
            try {
                Configuration configuration = new Configuration();
                // Hibernate settings equivalent to
hibernate.cfg.xml's properties
                Properties settings = new Properties();
                settings.put(Environment.DRIVER,
"oracle.jdbc.driver.OracleDriver");
                settings.put(Environment.URL,
"jdbc:oracle:thin:@localhost:1521:xe");
                settings.put(Environment.USER, "system");
                settings.put(Environment.PASS, "oracle");
                settings.put(Environment.DIALECT,
"org.hibernate.dialect.Oracle10gDialect");
                settings.put(Environment.SHOW_SQL, "true");

                settings.put(Environment.CURRENT_SESSION_CONTEXT_CLASS, "thread");
                settings.put(Environment.HBM2DDL_AUTO, "create-
drop");

                configuration.setProperties(settings);
                configuration.addAnnotatedClass(User.class);
                ServiceRegistry serviceRegistry = new
StandardServiceRegistryBuilder()

                .applySettings(configuration.getProperties()).build();
                System.out.println("Hibernate Java Config
serviceRegistry created");
                sessionFactory =
configuration.buildSessionFactory(serviceRegistry);
                return sessionFactory;
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    }
}

```

```

        }
    }
    return sessionFactory;
}
}

```

7. Create a UserDao.java

Let's create a *UserDAO* class which is a Data Access Layer (DAO) class that provides CRUD (Create, Read, Update, Delete) operations using for the **users table** in a database using Hibernate. Here's the full source code of the *UserDAO*:

```

package com.company.usermanagement.dao;

import java.util.List;

import com.company.usermanagement.model.User;
import com.company.usermanagement.util.HibernateUtil;

import org.hibernate.Session;
import org.hibernate.Transaction;

public class UserDao {
    /**
     * Save User
     * @param user
     */
    public void saveUser(User user) {
        Transaction transaction = null;
        try (Session session =
HibernateUtil.getSessionFactory().openSession()) {
            // start a transaction
            transaction = session.beginTransaction();
            // save the student object
            session.save(user);
            // commit transaction
            transaction.commit();
        } catch (Exception e) {
            if (transaction != null) {
                transaction.rollback();
            }
            e.printStackTrace();
        }
    }
}

```



```

/**
 * Update User
 * @param user
 */
public void updateUser(User user) {
    Transaction transaction = null;
    try (Session session =
HibernateUtil.getSessionFactory().openSession()) {
        // start a transaction
        transaction = session.beginTransaction();
        // save the student object
        session.update(user);
        // commit transaction
        transaction.commit();
    } catch (Exception e) {
        if (transaction != null) {
            transaction.rollback();
        }
        e.printStackTrace();
    }
}

/**
 * Delete User
 * @param id
 */
public void deleteUser(int id) {

    Transaction transaction = null;
    try (Session session =
HibernateUtil.getSessionFactory().openSession()) {
        // start a transaction
        transaction = session.beginTransaction();

        // Delete a user object
        User user = session.get(User.class, id);
        if (user != null) {
            session.delete(user);
            System.out.println("user is deleted");
        }

        // commit transaction
        transaction.commit();
    } catch (Exception e) {
        if (transaction != null) {
            transaction.rollback();
        }
    }
}

```

```

        }
        e.printStackTrace();
    }
}

/**
 * Get User By ID
 * @param id
 * @return
 */
public User getUser(int id) {

    Transaction transaction = null;
    User user = null;
    try (Session session =
HibernateUtil.getSessionFactory().openSession()) {
        // start a transaction
        transaction = session.beginTransaction();
        // get an user object
        user = session.get(User.class, id);
        // commit transaction
        transaction.commit();
    } catch (Exception e) {
        if (transaction != null) {
            transaction.rollback();
        }
        e.printStackTrace();
    }
    return user;
}

/**
 * Get all Users
 * @return
 */
@SuppressWarnings("unchecked")
public List < User > getAllUser() {

    Transaction transaction = null;
    List < User > listOfUser = null;
    try (Session session =
HibernateUtil.getSessionFactory().openSession()) {
        // start a transaction
        transaction = session.beginTransaction();
        // get an user object

```

```

        listOfUser = session.createQuery("from
User").getResultList();

        // commit transaction
        transaction.commit();
    } catch (Exception e) {
        if (transaction != null) {
            transaction.rollback();
        }
        e.printStackTrace();
    }
    return listOfUser;
}
}

```

8. Create a UserServlet.java

Now, let's create a *UserServlet* that acts as a page controller to handle all requests from the client.

Let's look at the code first:

```

package com.company.usermanagement.web;

import java.io.IOException;
import java.sql.SQLException;
import java.util.List;

import javax.servlet.RequestDispatcher;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import com.company.usermanagement.dao.UserDao;
import com.company.usermanagement.model.User;

/**
 * Servlet implementation class UserServlet
 */
@WebServlet("/")
public class UserServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    /**

```

```

    * @see HttpServlet#HttpServlet()
    */
    private UserDao userDao;

    public UserServlet() {
        super();
        // TODO Auto-generated constructor stub
    }

    public void init() {
        userDao = new UserDao();
    }

    /**
     * @see HttpServlet#doGet(HttpServletRequest request,
     HttpServletResponse response)
     */
    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {
        String action = request.getServletPath();

        try {
            switch (action) {
                case "/new":
                    showNewForm(request, response);
                    break;
                case "/insert":
                    insertUser(request, response);
                    break;
                case "/delete":
                    deleteUser(request, response);
                    break;
                case "/edit":
                    showEditForm(request, response);
                    break;
                case "/update":
                    updateUser(request, response);
                    break;
                default:
                    listUser(request, response);
                    break;
            }
        } catch (SQLException ex) {
            throw new ServletException(ex);
        }
    }

```

```

        }
    }

    /**
     * @see HttpServlet#doPost(HttpServletRequest request,
     *                          HttpServletResponse response)
     */
    protected void doPost(HttpServletRequest request,
        HttpServletResponse response) throws ServletException, IOException {
        // TODO Auto-generated method stub
        doGet(request, response);
    }

    private void listUser(HttpServletRequest request,
        HttpServletResponse response)
        throws SQLException, IOException, ServletException {
        List < User > listUser = userDao.getAllUser();
        request.setAttribute("listUser", listUser);
        RequestDispatcher dispatcher =
        request.getRequestDispatcher("user-list.jsp");
        dispatcher.forward(request, response);
    }

    private void showNewForm(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {
        RequestDispatcher dispatcher =
        request.getRequestDispatcher("user-form.jsp");
        dispatcher.forward(request, response);
    }

    private void showEditForm(HttpServletRequest request,
        HttpServletResponse response)
        throws SQLException, ServletException, IOException {
        int id = Integer.parseInt(request.getParameter("id"));
        User existingUser = userDao.getUser(id);
        RequestDispatcher dispatcher =
        request.getRequestDispatcher("user-form.jsp");
        request.setAttribute("user", existingUser);
        dispatcher.forward(request, response);
    }

    private void insertUser(HttpServletRequest request,
        HttpServletResponse response)

```

```

        throws SQLException, IOException {
            String name = request.getParameter("name");
            String email = request.getParameter("email");
            String country = request.getParameter("country");
            User newUser = new User(name, email, country);
            userDao.saveUser(newUser);
            response.sendRedirect("list");
        }

        private void updateUser(HttpServletRequest request,
            HttpServletResponse response)
            throws SQLException, IOException {
            int id = Integer.parseInt(request.getParameter("id"));
            String name = request.getParameter("name");
            String email = request.getParameter("email");
            String country = request.getParameter("country");

            User user = new User(id, name, email, country);
            userDao.updateUser(user);
            response.sendRedirect("list");
        }

        private void deleteUser(HttpServletRequest request,
            HttpServletResponse response)
            throws SQLException, IOException {
            int id = Integer.parseInt(request.getParameter("id"));
            userDao.deleteUser(id);
            response.sendRedirect("list");
        }
    }
}

```

9. Creating User Listing JSP Page - user-list.jsp

Next, create a JSP page for displaying all **users** from the database.

Let's create a *list-user.jsp* page under the **WebContent** directory in the project with the following code:

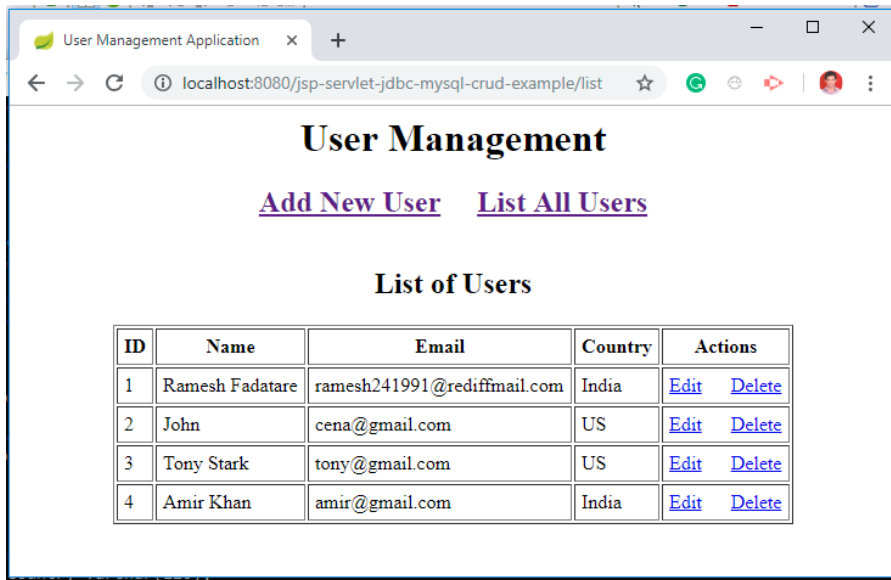
```

<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<html>
<head>
    <title>User Management Application</title>
</head>

```

[illegible]

Once you will deploy the above JSP page in tomcat and open it in the browser looks something like this:



10. Create a User Form JSP Page - user-form.jsp

Next, we create a JSP page for creating a new User called *user-form.jsp*. Here's its full source code:

[illegible]

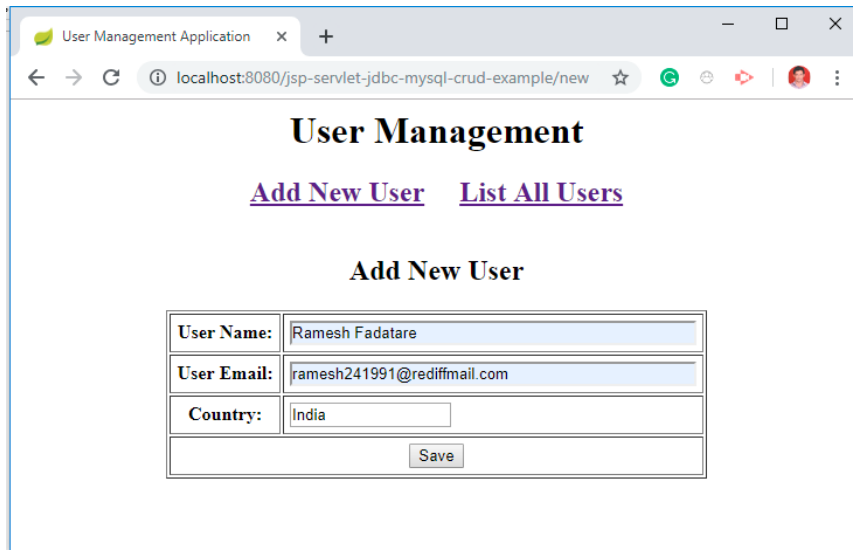

```

<table border="1" cellpadding="5">
  <caption>
    <h2>
      <c:if test="${user != null}">
        Edit User
      </c:if>
      <c:if test="${user == null}">
        Add New User
      </c:if>
    </h2>
  </caption>
  <c:if test="${user != null}">
    <input type="hidden" name="id" value="<c:out
value='${user.id}' />" />
  </c:if>
  <tr>
    <th>User Name: </th>
    <td>
      <input type="text" name="name" size="45"
        value="<c:out value='${user.name}' />"
      />
    </td>
  </tr>
  <tr>
    <th>User Email: </th>
    <td>
      <input type="text" name="email" size="45"
        value="<c:out value='${user.email}' />"
      />
    </td>
  </tr>
  <tr>
    <th>Country: </th>
    <td>
      <input type="text" name="country" size="15"
        value="<c:out value='${user.country}' />"
      />
    </td>
  </tr>
  <tr>
    <td colspan="2" align="center">
      <input type="submit" value="Save" />
    </td>
  </tr>
</table>
</form>

```

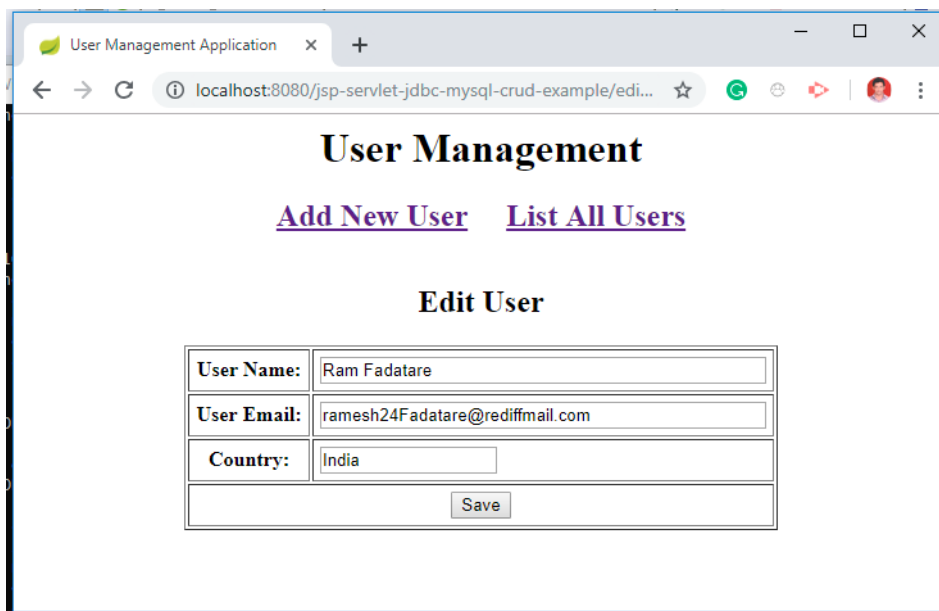
```
</div>
</body>
</html>
```

Once you will deploy the above JSP page in tomcat and open it in the browser looks something like this:



The screenshot shows a web browser window titled 'User Management Application'. The address bar shows 'localhost:8080/jsp-servlet-jdbc-mysql-crud-example/new'. The page content includes a title 'User Management', two links '[Add New User](#)' and '[List All Users](#)', and a section titled 'Add New User'. Below this is a form with three input fields: 'User Name' (containing 'Ramesh Fadatare'), 'User Email' (containing 'ramesh241991@rediffmail.com'), and 'Country' (containing 'India'). A 'Save' button is located at the bottom right of the form.

The above page acts for both functionalities to create a new User and Edit the same user. The edit page looks like this:



The screenshot shows a web browser window titled 'User Management Application'. The address bar shows 'localhost:8080/jsp-servlet-jdbc-mysql-crud-example/edi...'. The page content includes a title 'User Management', two links '[Add New User](#)' and '[List All Users](#)', and a section titled 'Edit User'. Below this is a form with three input fields: 'User Name' (containing 'Ram Fadatare'), 'User Email' (containing 'ramesh24Fadatare@rediffmail.com'), and 'Country' (containing 'India'). A 'Save' button is located at the bottom right of the form.

11. Creating Error JSP page

Here's the code of the *Error.jsp* page which simply shows the exception message:

```

<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8" isErrorPage="true" %>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
    "http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<title>Error</title>
</head>
<body>
    <center>
        <h1>Error</h1>
        <h2><%=exception.getMessage() %><br/> </h2>
    </center>
</body>
</html>

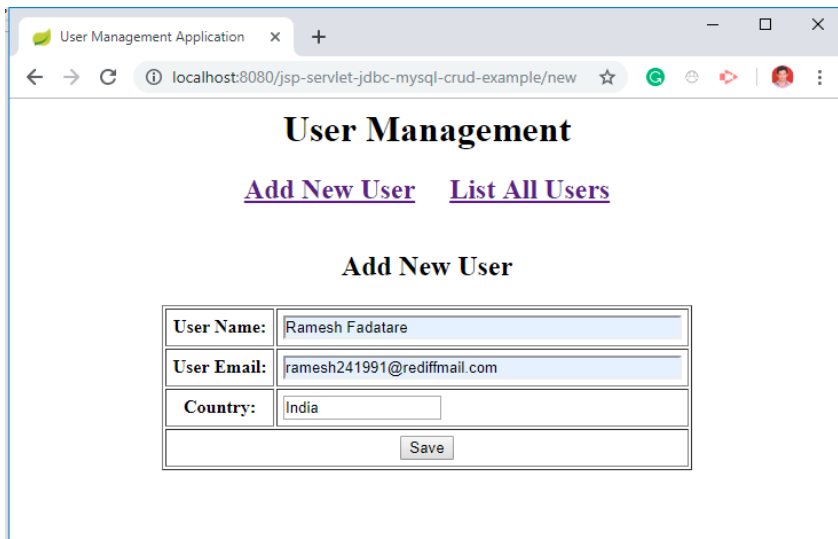
```

12. Deploying and Testing the Application

It's time to see a demo of the above **User Management** web application. Deploy this web application to the tomcat server.

Type the following URL in your web browser to access the **User Management** application: <http://localhost:8080/jsp-servlet-jdbc-mysql-crud-example/>

Create a new User



The screenshot shows a web browser window titled "User Management Application" with the address bar displaying "localhost:8080/jsp-servlet-jdbc-mysql-crud-example/new". The page content includes the title "User Management", two links "Add New User" and "List All Users", and a section titled "Add New User" containing a form. The form has three input fields: "User Name" with the value "Ramesh Fadatare", "User Email" with the value "ramesh241991@rediffmail.com", and "Country" with the value "India". A "Save" button is located at the bottom of the form.

User Name:	Ramesh Fadatare
User Email:	ramesh241991@rediffmail.com
Country:	India
Save	

Edit a User

User Management

[Add New User](#) [List All Users](#)

Edit User

User Name:	Ram Fadatare
User Email:	ramesh24Fadatare@rediffmail.com
Country:	India
Save	

List of all Users

User Management

[Add New User](#) [List All Users](#)

List of Users

ID	Name	Email	Country	Actions
1	Ramesh Fadatare	ramesh241991@rediffmail.com	India	Edit Delete
2	John	cena@gmail.com	US	Edit Delete
3	Tony Stark	tony@gmail.com	US	Edit Delete
4	Amir Khan	amir@gmail.com	India	Edit Delete

The source code for this tutorial (User Management) is available on my GitHub repository.

Link:: **G:\AdvancedJava\eclipse-workspace**

HB Generator classes

The <generator> class is a sub-element of id. It is used to generate the unique identifier for the objects of persistent class. There are many generator classes defined in the Hibernate Framework.

All the generator classes implements the org.hibernate.id.IdentifierGenerator interface. The application programmer may create one's own generator classes by implementing the IdentifierGenerator interface. Hibernate framework provides many built-in generator classes:

1. assigned
2. increment
3. sequence
4. hilo
5. native
6. identity
7. seqhilo
8. uuid
9. guid
10. select
11. foreign
12. sequence-identity

1) *assigned*

It is the default generator strategy if there is no <generator> element . In this case, application assigns the id. For example:

```
.....
<hibernate-mapping>
  <class ...>
    <id ...>
      <generator class="assigned"></generator>
    </id>

    .....

  </class>
</hibernate-mapping>
```

2) *increment*

It generates the unique id only if no other process is inserting data into this table. It generates short, int or long type identifier. If a table contains an identifier then the application considers its maximum value else the application considers that the first generated identifier is 1. For each attribute value, the hibernate increment the identifier by 1.

Syntax:

```
.....
<hibernate-mapping>
  <class ...>
    <id ...>
      <generator class="increment"></generator>
    </id>

    .....

  </class>
</hibernate-mapping>
```

3) *sequence*

It uses the sequence of the database. if there is no sequence defined, it creates a sequence automatically e.g. in case of Oracle database, it creates a sequence named HIBERNATE_SEQUENCE. In case of Oracle, DB2, SAP DB, Postgre SQL or McKoi, it uses sequence but it uses generator in interbase. Syntax:

```
.....
<id ...>
  <generator class="sequence"></generator>
</id>
.....
```

For defining your own sequence, use the param subelement of generator.

```
.....
<id ...>
  <generator class="sequence">
    <param name="sequence">your_sequence_name</param>
  </generator>
</id>
.....
```

4) *hilo*

It uses high and low algorithm to generate the id of type short, int and long. Syntax:

```
.....
<id ...>
  <generator class="hilo"></generator>
```

```
</id>
```

```
.....
```

5) native

It uses identity, sequence or hilo depending on the database vendor. Syntax:

```
.....
```

```
<id ...>
```

```
<generator class="native"></generator>
```

```
</id>
```

```
.....
```

6) identity

It is used in Sybase, My SQL, MS SQL Server, DB2 and HypersonicSQL to support the id column. The returned id is of type short, int or long. It is responsibility of database to generate unique identifier.

7) seqhilo

It uses high and low algorithm on the specified sequence name. The returned id is of type short, int or long.

8) uuid

It uses 128-bit UUID algorithm to generate the id. The returned id is of type String, unique within a network (because IP is used). The UUID is represented in hexadecimal digits, 32 in length.

9) guid

It uses GUID generated by database of type string. It works on MS SQL Server and MySQL.

10) select

It uses the primary key returned by the database trigger.

11) foreign

It uses the id of another associated object, mostly used with <one-to-one> association.

12) sequence-identity

It uses a special sequence generation strategy. It is supported in Oracle 10g drivers only.

HB Dialects

Hibernate supports multiple SQL dialects to interact with different databases. The SQL dialect in Hibernate determines the specific SQL syntax and features used for generating and executing SQL statements.

Here are some commonly used SQL dialects in Hibernate:

1. MySQL:

```
hibernate.dialect = org.hibernate.dialect.MySQLDialect
```

2. Oracle:

```
hibernate.dialect = org.hibernate.dialect.OracleDialect
```

3. PostgreSQL:

```
hibernate.dialect = org.hibernate.dialect.PostgreSQLDialect
```

4. Microsoft SQL Server:

```
hibernate.dialect = org.hibernate.dialect.SQLServerDialect
```

5. H2 Database:

```
hibernate.dialect = org.hibernate.dialect.H2Dialect
```

6. SQLite:

```
hibernate.dialect = org.hibernate.dialect.SQLiteDialect
```

7. DB2:

```
hibernate.dialect = org.hibernate.dialect.DB2Dialect
```

These are just a few examples of SQL dialects supported by Hibernate. The appropriate SQL dialect for your application depends on the database you are using. You should choose the dialect that matches your database system to ensure that Hibernate generates SQL statements compatible with that particular database.

In Hibernate configuration files, you can specify the SQL dialect by setting the `hibernate.dialect` property to the fully qualified name of the dialect class. This property can be set either in the Hibernate configuration file (*hibernate.cfg.xml*) or in a Java properties file.

Using the correct SQL dialect in Hibernate ensures that the generated SQL statements are optimized for the specific database, taking advantage of its features and syntax.